

Algoritmi e strutture dati - esame del 12/6/2019

Svolgimenti

Cognome e nome: _____ Matr. _____

Esercizio 1. (Punti 3) Sia $f(n)$ una funzione non decrescente e positiva per qualunque n . Si dimostri oppure si refuti ognuna delle seguenti implicazioni:

- (1) $f(n) \in O(g(n)) \implies 2f(n) + 12 \in O(g(n))$
- (2) $f(n) \in O(g(n)) \implies n \cdot f(n) - 10 \in O(g(n))$
- (3) $f(n) \in O(g(n)) \implies f(n)/n + 120 \in O(g(n))$

Nota: mentre una dimostrazione deve valere per qualunque f (positiva e non decrescente) e g tali che $f(n) \in O(g(n))$, la refutazione di un'implicazione può basarsi su una specifica scelta delle funzioni f, g , tali per cui sia vero l'antecedente ma non il conseguente dell'implicazione refutata.

Soluzione 1. (1) è vera: per l'algebra di O -grande si ha $2f(n) + 12 \in O(f(n))$, ossia $2f(n) + 12 \leq c \cdot f(n)$ per qualche $c > 0$ e q.o.; per ipotesi $f(n) \leq d \cdot g(n)$ per qualche $d > 0$ e q.o. dunque:

$$2f(n) + 12 \leq c \cdot d \cdot g(n) \quad \text{q.o.}$$

(2) è falsa in generale: si prendano $f(n) = g(n) = n$, allora trivialmente $n \in O(n)$, ma $\lim_{n \rightarrow \infty} (n^2 - 10)/n = \infty$, da cui discende che $n^2 - 10 \notin O(n)$.

(3) è vera: poiché n ed $f(n)$ sono non negative e $n \neq 0$ (altrimenti il rapporto non è definito), $f(n)/n \leq f(n)$ per ogni $n > 0$, da cui segue che $O(f(n)/n) \subseteq O(f(n))$. Visto che $f(n)/n + 120 \in O(f(n)/n)$ e che per ipotesi $O(f(n)) \subseteq O(g(n))$, segue la tesi.

Esercizio 2. (Punti 6) Supponendo che la funzione $\text{FOO}(n)$ richieda tempo $\Theta(n)$, si analizzi la complessità temporale in funzione di n del seguente algoritmo, ricavando la relazione di ricorrenza e calcolandone il Θ senza far ricorso al master theorem:

```
GOLDRAKE( $n$ )
  Pre:  $n \geq 1$  intero
  if  $n = 1 \vee n = 2$  then
    return 1
  else
    temp  $\leftarrow$  GOLDRAKE( $n - 2$ ) + FOO( $2n$ )
    return temp
```

Soluzione 2. La relazione di ricorrenza di GOLDRAKE è $T(n) = \Theta(1)$ se $n \leq 2$, $T(n) = T(n - 2) + 2n$ altrimenti. Svolgendo si ha:

$$T(n) = T(n - 2) + 2n = T(n - 4) + 2(n - 2) + 2n = T(n - 6) + 2(n - 4) + 2(n - 2) + 2n$$

quindi in generale:

$$T(n) = T(n - 2k) + 2 \sum_{i=0}^{k-1} (n - 2i) \quad k \leq n/2$$

Posto $k = n/2$ otteniamo (a meno del termine costante):

$$T(n) = 2 \sum_{i=0}^{n/2-1} (n-2i) + 1 = 2 \left(\sum_{i=0}^{n/2-1} n - 2 \sum_{i=0}^{n/2-1} i \right) + 1 = 2 \left(\frac{n^2}{2} - \frac{n^2}{4} + \frac{n}{2} \right) + 1 = \frac{1}{2}n^2 + n + 1 \in \Theta(n^2).$$

Esercizio 3. (Punti 7) Si consideri il seguente algoritmo:

```
GEZA(A[1..n])
  for i ← n down to 2 do
    k ← i
    for j ← 1 to i - 1 do
      if A[j] < A[k] then
        k ← j
    scambia A[i] con A[k]
  return A
```

- Cosa fa GEZA?
- Qual è l'invariante del ciclo esterno?
- Qual è la complessità di GEZA in termini di Θ ?
- Sapreste indicare un algoritmo che risolva lo stesso problema in tempo asintoticamente migliore?

Soluzione 3. GEZA ordina $A[1..n]$ in modo non crescente (quindi decrescente se tutti gli elementi sono distinti). Dopo l'esecuzione del ciclo più interno avremo che $A[k] = \min A[1..i]$, quindi l'invariante del ciclo esterno risulta:

$$\forall x \in A[1..i] \forall y \in A[i+1..n]. x \geq y \quad \& \quad \forall j \in i+1..n-1. A[j] \geq A[j+1]$$

ossia tutti gli elementi di $A[1..i]$ sono maggiori o uguali di quelli in $A[i+1..n]$ e quest'ultimo semivettore è ordinato in modo non crescente.

Trattandosi di una variante del SELECTSORT, la complessità di GEZA è $\Theta(n^2)$. Un'opportuna variante del MERGESORT o dello HEAPSORT risolvono il problema in tempo $\Theta(n \log n)$.

Esercizio 4. (Punti 4)

- Si riporti la definizione di albero binario di ricerca con chiavi intere.
- Si consideri la ricerca di una chiave in un albero binario di ricerca con n nodi. Considerando la possibile forma dell'albero si individuino i casi in cui la ricerca abbia complessità massima o minima, sempre con riferimento al caso peggiore della ricerca di una chiave in un dato albero. Si giustifichi la risposta.

Soluzione 4. Un albero binario con chiavi intere è di ricerca se vuoto oppure se entrambi i sottoalberi sinistro e destro sono di ricerca, e se la chiave in radice è maggiore di tutte quelle nel sottoalbero sinistro e minore di tutte quelle nel sottoalbero destro.

La complessità della ricerca in un albero di ricerca è sempre $\Theta(h)$ dove h è l'altezza dell'albero. Il caso di complessità massima è quello di un albero costituito da un solo ramo; in tal caso la ricerca è $\Theta(n)$, essendo $h = n - 1$. Il caso di complessità minima è quello di un albero completo almeno sino al penultimo livello, in cui la complessità risulta $\Theta(\log n)$ essendo $(\log_2 n) - 1 \leq h \leq \log_2 n$.

Esercizio 5. (Punti 6) Sia dato un albero k -ario T non vuoto, rappresentato con puntatori *child* e *sibling*. Si dia un algoritmo SHORTEST(T), dove $T \neq nil$ punta alla radice, che restituisca il numero di nodi lungo il ramo più corto dell'albero. (Un ramo è un cammino dalla radice ad una foglia. Una foglia è un nodo che non ha figli.)

Soluzione 5. Il problema si può risolvere con una DFS dell'albero:

SHORTEST(T)

```

Pre:  $T \neq nil$ 
if  $T.child = nil$  then return 1
else
   $m \leftarrow \text{SHORTEST}(T.child)$ 
   $C \leftarrow T.child.sibling$ 
  while  $C \neq nil$  do
     $m \leftarrow \min(m, \text{SHORTEST}(C))$ 
     $C \leftarrow C.sibling$ 
  return  $m + 1$ 

```

Una soluzione alternativa (che qui non illustriamo) consiste in una BFS modificata per il calcolo delle distanze, che ritorni il minimo delle distanze delle foglie dalla radice più 1.

Esercizio 6. (Punti 5) Si riporti l'algoritmo che effettua l'estrazione del minimo da uno heap a minimo.

Soluzione 6.

```

HEAPEXTRACTMIN( $H$ )
Pre:  $H$  heap minimo,  $H.size > 0$ 
 $result \leftarrow H[1]$ 
 $H[1] \leftarrow H[H.size]$ 
 $H.size \leftarrow H.size - 1$ 
 $p \leftarrow 1$ 
while  $p < H.size$  do
  if  $2p \leq H.size$  &  $H[2p] < H[p]$  then
     $minimo \leftarrow 2p$ 
  else
     $minimo \leftarrow p$ 
  if  $2p + 1 \leq H.size$  &  $H[2p + 1] < H[minimo]$  then
     $minimo \leftarrow 2p + 1$ 
  if  $minimo \neq p$  then
    scambia  $H[minimo]$  e  $H[p]$ 
     $p \leftarrow minimo$ 
  else
    esci dal ciclo
return  $result$ 

```

Esercizio 7. (Punti 5) Si determini un ordinamento topologico del seguente grafo aciclico, basandosi sulla versione temporizzata della visita in profondità. Si marchi ogni vertice con i suoi tempi di “inizio visita” e di “fine visita”, e si riporti la sequenza dei vertici prodotta dall'algoritmo.

```

 $a : c, e, f$ 
 $b : d, a, c$ 
 $c : h$ 
 $d : c, h, e$ 
 $e : f$ 
 $f : h$ 
 $h :$ 

```

Soluzione 7. La visita DFS che parte da a e che sceglie i vertici per come sono elencati sia nelle liste di adiacenza che nel vettore (verticale) dei vertici, produce i seguenti tempi:

$a(1/10), c(2/5), h(3/4), e(6/9), f(7/8), b(11/14), d(12/13).$

Ordinando i vertici in modo decrescente rispetto ai tempi di fine visita si ottiene l'ordinamento topologico:

$b(14), d(13), a(10), e(9), f(8), c(5), h(4).$